

```

import crypto from 'node:crypto';
import { q, one, j } from '../db.js';
import { resolve } from './providers.js';
import { listCapabilities } from './policy.js';
import { listPackages, registerGenerated } from './registry.js';
import { route } from './router.js';
import { install } from './installer.js';
import { sign } from './signing.js';

// Describe what you want. The bound builder composes a plan out of what the
// platform can already do. Nothing is created until someone approves it.
export async function propose({ ctx, intent }) {
  const builder = await resolve({ slot: 'builder', businessId: ctx.businessId });
  if (!builder) throw new Error('no builder provider bound');

  const think = (task) => route({ ctx, task });
  const plan = await builder.module.plan({
    intent,
    capabilities: listCapabilities(),
    packages: listPackages(),
    think,
  });

  return one(
    `insert into build_plan (business_id, requested_by, intent, plan)
    values ($1,$2,$3,$4::jsonb) returning *`,
    [ctx.businessId, ctx.person?.id ?? null, intent, JSON.stringify(plan)]
  );
}

// Approving turns the plan into a signed package and installs it for this
// business only. It goes through the same installer as anything else.
export async function approve({ ctx, planId }) {
  const row = await one(
    `select * from build_plan where id = $1 and business_id = $2`, [planId, ctx.b
  );
  if (!row) throw new Error('no such plan');
  if (row.state !== 'proposed') throw new Error(`plan is ${row.state}`);

  const plan = j(row.plan);
  const manifest = plan.manifest;

  const mod = registerGenerated(manifest);
  await sign({ packageKey: manifest.key, version: manifest.version,

```

```

        contentHash: hashOf(manifest), signer: `business:${ctx.business.sl
        trustTier: 'community' });

await one(
  `insert into package (key, version, kind, name, summary, manifest, source, co
  values ($1,$2,$3,$4,$5,$6::jsonb,'builder',$7)
  on conflict (key, version) do update set manifest = excluded.manifest return
  [manifest.key, manifest.version, manifest.kind, manifest.name,
  manifest.summary ?? null, JSON.stringify(manifest), hashOf(manifest)]
);

await install({ businessId: ctx.businessId, packageKey: manifest.key, grantTo:

await q(
  `update build_plan set state = 'deployed', produced_package = $1, decided_at
  [manifest.key, planId]
);

return { packageKey: manifest.key, capabilities: mod.capabilities.map(c => c.ke
}

export async function reject({ ctx, planId }) {
  return one(
    `update build_plan set state = 'rejected', decided_at = now()
    where id = $1 and business_id = $2 returning *`, [planId, ctx.businessId]
  );
}

function hashOf(manifest) {
  return crypto.createHash('sha256').update(JSON.stringify(manifest)).digest('hex
}

```